

EEET2588

Real-time Systems Engineering & Design

Lecture 4: Data Protection and Synchronization

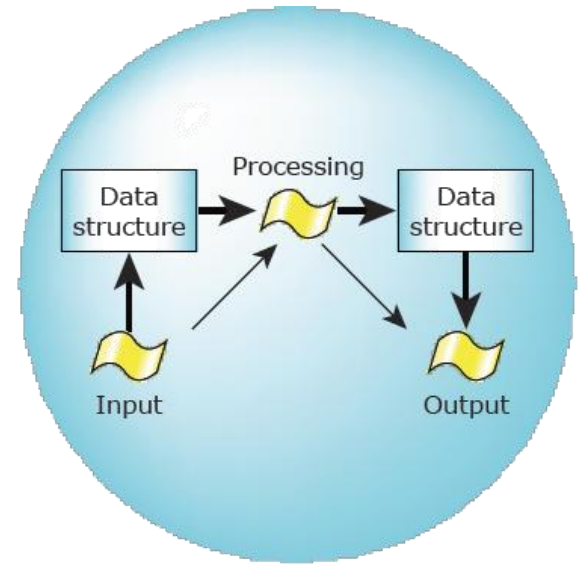
- Mutex, Barriers, Read/Write Locks, Sleep-on Lock, Condvars, Semaphores and Message Passing
- Sequence Diagrams

Dr Linh Tran

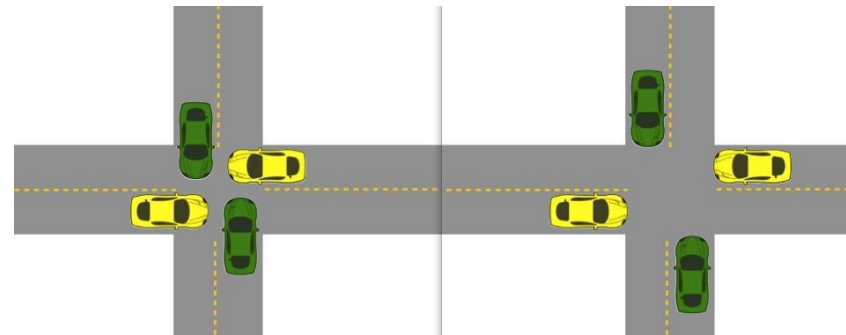
Course note acknowledgment: Original course notes developed by Roy Ferguson and Samuel Ippolito from RMIT Melbourne

Lecture 3 Review

- Showed why it is important to protect shared data
 - Producer-Consumer Example
 - Atomic operations
 - Critical section
 - Race Conditions
- Safety and Liveness and their relationship with:
 - Deadlock, Starvation & Livelock
- Mutual Exclusion
 - Peterson's Algorithm
 - Test and Set Algorithm
 - QNX primitive – `pthread_mutex_lock()`



Multiple operations, multiple threads [1]



Example of Livelock... no meaningful progress [2]

Lecture 4 Overview

- Protect the critical sections of the code by using **Synchronization Primitive**
 - The below operations enable you to safely block other process from accessing critical data section until they are released -> Example analysis.
 - And what are the differences between them?

- **Mutex**
- **Barriers**
- **Read/write locks**
- **Sleepon Locks**
- **Conditional variables**
- **Semaphores**
- **Message passing**

- Sequence Diagram examples
- QNX Periodic Timers

Used in Lab 3	Synchronization service		Supported between processes	Supported across a QNX Neutrino LAN
➔	Mutexes	(p. 47)	Yes (with shared <u>mem</u>)	No
➔	<u>Condvars</u>	(p. 48)	Yes (with shared <u>mem</u>)	No
➔	Barriers	(p. 49)	No	No
	<u>Sleepon locks</u>	(p. 51)	No	No
➔	Reader/writer locks	(p. 51)	Yes	No
➔	Semaphores	(p. 52)	Yes	Yes (named)
	FIFO scheduling	(p. 53)	Yes	No
	Send/Receive/Reply	(p. 53)	Yes	Yes (named)
	Atomic operations	(p. 53)	Yes	No

- **Note:** The [QNX_Neutrino_RTOS_System_Architecture.pdf](#) [1] has a good summary of the available synchronization services (starting at page 46). Furthermore, information on QNX functions can be found by typing name in to the search box in the following link:
<https://www.qnx.com/developers/docs/7.0.0/>

Mutual Exclusion

- Mutual exclusion lock, or *mutex*, is the **simplest** of the synchronization services. Called a *mutex* (an acronym for **MUTual EXclusion**).

- A mutex is typically acquired by using:

```
#include <pthread.h>

int pthread_mutex_lock( pthread_mutex_t* mutex );
```

- To obtain lock within a given time use the timed version:

```
#include <pthread.h>
#include <time.h>

int pthread_mutex_timedlock( pthread_mutex_t * mutex,
                             const struct timespec * abs_timeout );
```

- **abs_timeout** is a pointer to a **timespec** struct that specifies the maximum time to wait to lock the mutex, expressed as an absolute time.
- There is a monotonic version of the function that uses the **CLOCK_MONOTONIC** clock, which ensures that the timeout isn't affected by changes to the system time.

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.lib_ref/topic/p/pthread_mutex_lock.html

Mutual Exclusion – Bathroom example in code

Simple example showing that only one thread can use a shower (resource) at a time.

- If the mutex is already locked, then the waiting thread block.
- The thread with the highest priority then can acquire the mutex. If all the same priority they acquire in FIFO order.

```
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <errno.h>

#define N_ex 4 // #of threads

pthread_mutex_t someoneisbathing;
```

```
void *shower_ex(void *notused)
```

```
{
    pthread_mutex_lock(&someoneisbathing);
```

```
    printf("Thread %d is using shower\n", pthread_self());
    sleep(2);
    printf("Thread %d is leaving shower\n", pthread_self());
```

```
    pthread_mutex_unlock(&someoneisbathing);
}
```

```
void main( void )
{
    printf("Bathroom Example\n\n");
    pthread_mutex_init(&someoneisbathing, NULL);

    pthread_t *thread_ids; // Create thread ID ptr
    thread_ids = malloc (sizeof (pthread_t) * N_ex);

    int i;

    for(i=0; i < N_ex; i++) // launch threads
        pthread_create(&thread_ids[i], NULL, shower_ex, NULL);

    for(i=0; i < N_ex; i++) // Wait to join all threads
        pthread_join(thread_ids[i], NULL);

    printf("Main Terminated...\n");
}
```


Mutual Exclusion – Obtaining a Resource Within a Time Limit

Only one thread can use a shower (resource) at a time.

- If the mutex is already locked, then the calling thread blocks until it has acquired the mutex – unless it times out (in this case 1 second)

```
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <errno.h>
#include <string.h> // needed for QNX 700

pthread_mutex_t someoneisbathing;
struct timespec t_deadline;

void *shower_ex(void *notused) {
    int ret_code = pthread_mutex_timedlock(&someoneisbathing, &t_deadline);
    if(ret_code == EOK)
    { // this is the critical section:
        printf("Thread %d is using shower\n", pthread_self());
        sleep(2);
        printf("Thread %d is leaving shower\n", pthread_self());
        pthread_mutex_unlock(&someoneisbathing);
    }
    else printf("Thread %d Lock Failed: %s\n", pthread_self(), strerror(ret_code));
}
```

```
void main( void )
{
    printf("Bathroom Example - Timed Mutex\n\n");
    pthread_mutex_init(&someoneisbathing, NULL);
    pthread_t th1, th2;

    // get time
    clock_gettime(CLOCK_REALTIME, &t_deadline);
    // make the deadline 1 second from now
    t_deadline.tv_sec += 1;
    pthread_create(&th1, NULL, shower_ex, NULL);
    pthread_create(&th2, NULL, shower_ex, NULL);

    pthread_join(th1, NULL);
    pthread_join(th2, NULL); // th2 will fail
                             // to get mutex

    printf("Main Terminated...\n");
}
```

Barrier

- Barriers can be used to Synchronize participating threads
 - The calling thread blocks when calling `pthread_barrier_wait()` until the required number of threads have called `pthread_barrier_wait()`, specifying the barrier.
 - This allows a given number of threads to rendezvous at a given point.
- The barrier must first be initialised by calling `pthread_barrier_init()` with the third parameter being the number of threads that will participate in the barrier.

```
#include <pthread.h>

int pthread_barrier_init( pthread_barrier_t *barrier,
                        const pthread_barrierattr_t *attr,
                        unsigned int count );
```

- Each participating thread, when it reaches the desired synchronization point in its code will call `pthread_barrier_wait()`, which blocks the thread until all threads have called `pthread_barrier_wait()`. Then all the threads will resume execution.

```
int pthread_barrier_wait( pthread_barrier_t * barrier );
```

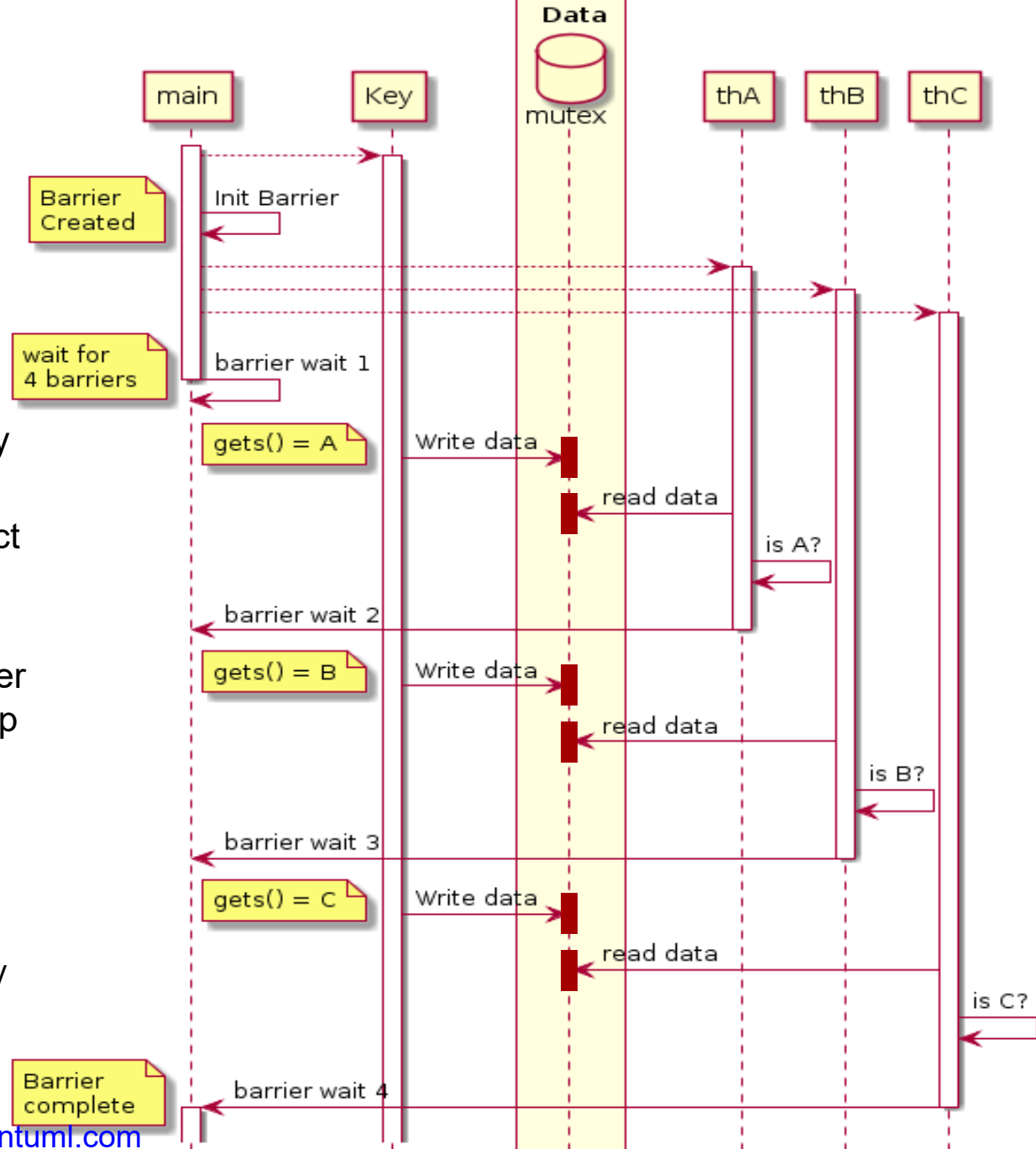
- A barrier can be destroyed at anytime, which will releases any resources used by the barrier. All threads will pass the barrier like it was never there.

```
int pthread_barrier_destroy( pthread_barrier_t * barrier );
```

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_procs_barriers.html

Barrier Example

- Example contain 5 threads
 - Main,
 - Key board scanner, &
 - Three reader threads
- Each reader thread waits for a character to be placed in the shared data variable by the key scanner
 - If the character is the correct one the thread calls `pthread_barrier_wait()`
 - If it isn't the correct character the thread continues to keep reading the shared data
- To ensure safety, shared data must be accessed with mutex
- Note: Sequence diagram does not show blocking of threads caused by mutual exclusive access to shared data or when the `gets()` function blocks while reading keyboard.



Sequence Diagram generated with plantuml.com

Barrier Example - code

RMIT Classification: Trusted

Example starts 3 threads that all need to get the correct input before the barrier is satisfied.

```
#include <stdlib.h>
#include <pthread.h>

char key = 0;
pthread_barrier_t barrier; // Barrier
pthread_mutex_t mutex = // Mutex
    PTHREAD_MUTEX_INITIALIZER;

void *thread_ex(void *data) {
    char ch = *((char*)(data));
    int loop=1, ID=pthread_self();
    printf("Th %i: waiting for '%c'\n", ID, ch);
    while(loop)
    {
        pthread_mutex_lock(&mutex);
        if(key==ch)
        {
            // reading shared data
            printf("Th %i got '%c' ", ID, ch);
            loop=0;
        }
        pthread_mutex_unlock(&mutex);
        sched_yield();
    }
    printf("--> Waiting at barrier\n", ID);
    pthread_barrier_wait(&barrier);
}
```

```
void * getkey_ex(void *notused) {
    char buf[2]={};
    int i = 0;
    while(1) {
        gets(buf); // read keyboard
        pthread_mutex_lock(&mutex);
        key = buf[0]; // writing data
        pthread_mutex_unlock(&mutex);
        sched_yield();
    }
}
```

```
void main( void ) {
    printf(" Barrier Example\n");
    printf(" Press keys: A B C\n\n");

    pthread_t th_Key; // start key scan thread
    pthread_create(&th_Key, NULL, getkey_ex, NULL);

    pthread_t thA, thB, thC;

    // create a barrier with a count of 4
    pthread_barrier_init(&barrier, NULL, 4);

    char c[]={ 'A', 'B', 'C' };
    pthread_create(&thA, NULL, thread_ex, &c[0]);
    pthread_create(&thB, NULL, thread_ex, &c[1]);
    pthread_create(&thC, NULL, thread_ex, &c[2]);

    printf("-->waiting at main barrier\n");
    pthread_barrier_wait(&barrier); // wait here
    printf("Main Terminated...\n");
}
```

Read / Write Locks

- “Multiple readers, single writer locks”.
- Firstly a read/write lock needs to be initialised:

```
#include <pthread.h>
```

```
int pthread_rwlock_init( pthread_rwlock_t * rwl, const pthread_rwlockattr_t * attr );
```

- `rwl` is a pointer to the r/w object
- `attr` is NULL, or a pointer to attribute struct initialized with `pthread_rwlockattr_init()`

- The following functions are used to acquire and release r/w locks:

```
pthread_rwlock_rdlock()
```

- Acquire a shared **read** lock on a r/w lock

```
pthread_rwlock_tryrdlock()
```

- Attempt to acquire a shared lock on a r/w lock

```
pthread_rwlock_wrlock()
```

- Acquire an exclusive **write** lock on a r/w lock

```
pthread_rwlock_trywrlock()
```

- Attempt to acquire an exclusive lock

```
pthread_rwlock_unlock()
```

- **Unlock** a read-write lock

```
pthread_rwlock_destroy()
```

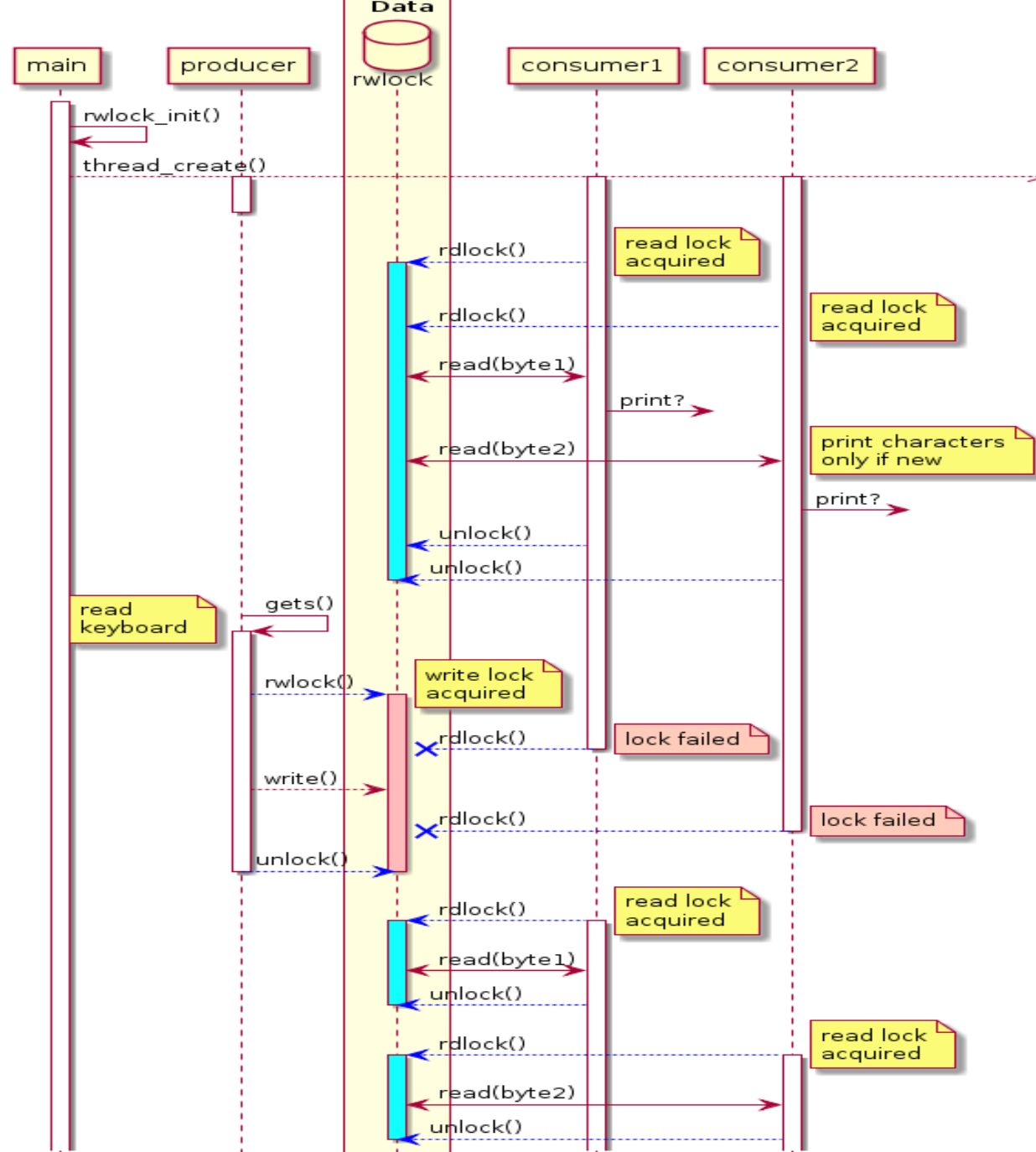
- **Destroy** a read-write lock

- What is the difference between **Acquire** and **Attempt to acquire**?
- Beware of **priority inversion** when using read-write locks. A high-priority thread may be blocked waiting on a read-write lock locked by a lower-priority thread.

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_procs_rwlock.html

Read / Write Lock Example

- Example shows:
 - 1 writer (producer) thread
 - 2 readers (consumers) threads
 - Thread1 consumes 1st character
 - Thread2 consumes 2nd character
 - All other characters are ignored
- Reader threads can access shared data as long as no write lock has been initiated.
- Once a write lock has been initiated only the writer can access the shared data variables.
 - Reader threads will block until write lock is unlocked.



Sequence Diagram
generated with plantuml.com

Read / Write Lock Example

Example starts 3 threads:

- 1 writer (producer) thread
- 2 readers (consumers) threads
- The readers read the shared data structure once every second
 - Could technically read the shared data at the same time – Should not modify it!
 - Each reader will only print to the screen if the data has changed value from the last time the thread accessed the shared data value.
- The writer reads to characters from the keyboard and stores the values in the shared data structure.

```
void main( void )
{
    // initialize read/write lock object
    pthread_rwlock_init(&dataSource, NULL);

    pthread_t th1, th2, th3;
    pthread_create(&th1, NULL, consumer1_ex, NULL);
    pthread_create(&th2, NULL, consumer2_ex, NULL);
    pthread_create(&th3, NULL, producer_ex, NULL);

    pthread_join(th1, NULL); // join reader 1
    pthread_join(th2, NULL); // will never get here

    printf("Main Terminated...\n");
}
```

Read / Write Lock Example

1 writer (producer) thread

2 readers (consumers) threads

```
#include <stdlib.h>
#include <pthread.h>
#include <errno.h>

pthread_rwlock_t dataSource;

struct resource {
    volatile char byte1;
    volatile char byte2;
} data;

void *producer_ex(void *notused)
{
    printf("Producer Started....\n");
    char buf[3]={};
    printf("press 2 keys then enter:\n");
    while(1)
    {
        gets(buf); // read keyboard
        pthread_rwlock_wrlock(&dataSource);
        data.byte1 = buf[0]; // key value
        data.byte2 = buf[1]; // modified
        pthread_rwlock_unlock(&dataSource);
    }
}
```

```
void *consumer1_ex(void *notused) {
    printf("Consumer1 Started....\n");
    char current=0, new=0;
    while(1) {
        pthread_rwlock_rdlock(&dataSource);
        new = data.byte1; //read data
        pthread_rwlock_unlock(&dataSource);
        if(new!=current) {
            printf("Consumer1 got: %c\n", new);
            current = new; //so we don't print same
            //char over and over again
        }
        sleep(1);
    }
}

void *consumer2_ex(void *notused) {
    printf("Consumer2 Started....\n");
    char current=0, new=0;
    while(1) {
        pthread_rwlock_rdlock(&dataSource);
        new = data.byte2; //read data
        pthread_rwlock_unlock(&dataSource);
        if(new!=current) {
            printf("Consumer2 got: %c\n", new);
            current = new; //so we don't print same
            //char over and over again
        }
        sleep(1);
    }
}
```

Sleepon Locks

- Sleepon locks can be used to block until a condition becomes true (like a memory location changing value) – can be used as a **simple conditional variable** (see next)
 - Uses an inbuilt mutual exclusion
 - Useful for letting a thread know that “data is now available”
- A thread (or several threads) can be blocked until an event happens by calling

```
#include <pthread.h>

int pthread_sleepon_wait( const volatile void * addr );
```

- **addr** is typically the data structure of the resource (just needs to be unique address)
- The **pthread_sleepon_lock()** function calls **pthread_mutex_lock()** on a mutex associated with the **pthread_sleepon** class of functions to protect the **critical section**:

```
int pthread_sleepon_lock( void );    // the return value should be checked
// critical section
int pthread_sleepon_unlock( void );
```

- The thread producing the event (producer) then calls either of these functions (during its critical section) to wake up the blocked threads (consumers):

```
int pthread_sleepon_signal( const volatile void * addr );    or
int pthread_sleepon_broadcast( const volatile void * addr );
```

- One of the consumer thread will own the lock at a time if **signal** is used
- **Broadcast** will wake up all thread, so it might be necessary to block a thread again.

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_procs_sleepon.html

Sleepon Locks – Used in Loops (simplest version)

Two consumer threads sleep until the producer threads creates some data

- Except this time, the **signal** is used on a common variable and the second consumer thread waits until the first thread completes before it is able to get the lock

- Only 1 thread will own the lock at a time

- If **broadcast** is used, both consumer threads will get the same data

```
struct resource {
    volatile char byte, notused; } data;

void *producer_ex(void *notused) {
    char buf[3]={};
    while(1) {
        printf("Type a char (a-z):\n");
        strcpy( buf, "  "); // clear buffer
        gets(buf);           // read keyboard
        pthread_sleepon_lock();
        data.byte = buf[0]; // save value

        // send signal to first blocked thread
        pthread_sleepon_signal( &data );
        //pthread_sleepon_broadcast(&data);
        pthread_sleepon_unlock();
    } // end while
}
```

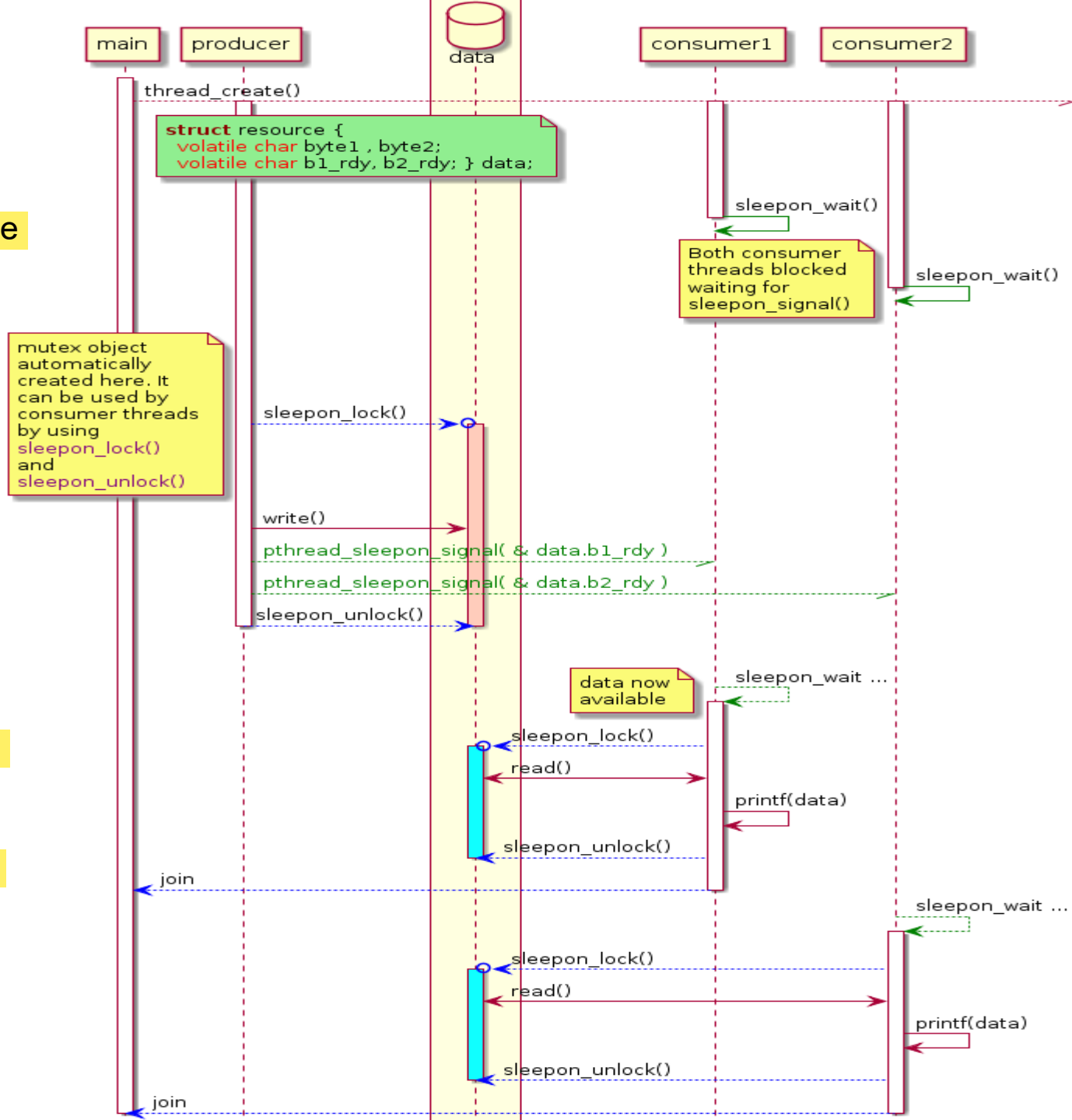
```
void *consumer1_ex(void *notused) {
    while(1) {
        pthread_sleepon_lock();
        pthread_sleepon_wait( &data );
        printf("Consumer 1 got: %c\n", data.byte);
        pthread_sleepon_unlock();
    }
}
```

```
void *consumer2_ex(void *notused) {
    while(1) {
        pthread_sleepon_lock();
        pthread_sleepon_wait( &data );
        printf("Consumer 2 got: %c\n", data.byte);
        pthread_sleepon_unlock();
    }
}
```

```
void main( void ) {
    pthread_create(NULL, NULL, consumer1_ex, NULL);
    pthread_create(NULL, NULL, consumer2_ex, NULL);
    pthread_create(NULL, NULL, producer_ex, NULL);
    sleep(20); // let threads run
    printf("Main Terminated...\n"); }
```

Sleepon Lock Example 2

- Consumer thread block and wait for a signal to be sent by the producer before attempting to access the shared data variables
- Producer thread creates a mutex lock, then modifies the data and unlocks the mutex after sending a signal to all the waiting threads
- The waiting threads (consumer threads) use `sleepon_lock()` and `sleepon_unlock()` to obtain a mutex on the shared data.



Sleepon Locks – Example 2

Two consumer threads sleep until the producer threads creates some data:

```
#include <stdlib.h>
#include <pthread.h>
#include <errno.h>

struct resource {
    volatile char byte1, byte2;
    volatile char b1_rdy, b2_rdy; } data;

void *producer_ex(void *notused) {
    char buf[3]={};
    printf("Type 2 chars (a-z):\n");
    strcpy( buf, "  "); // clear buffer
    gets(buf);           // read keyboard
    if( pthread_sleepon_lock() == EOK ) {
        data.byte1 = buf[0]; // key value
        data.byte2 = buf[1]; // modified
        data.b1_rdy = 1;     // make true
        data.b2_rdy = 1;     // make true
        // send signal to blocked threads
        pthread_sleepon_signal(&data.b1_rdy);
        pthread_sleepon_signal(&data.b2_rdy);
        // pthread_sleepon_broadcast(&data);
        pthread_sleepon_unlock();
    } // end if
}
```

```
void *consumer1_ex(void *notused) {
    if(pthread_sleepon_lock()==EOK) {
        while(!data.b1_rdy) {
            printf("Consumer 1 going to sleep\n");
            pthread_sleepon_wait( &data.b1_rdy );
            printf("Consumer 1 - exiting sleep\n");
        } // end inner while
        printf("Consumer1 got: %c\n", data.byte1);
        pthread_sleepon_unlock();
    } // end if
    else printf("Sleeplock ERROR in C1\n");
}
```

```
void *consumer2_ex(void *notused) {
    if(pthread_sleepon_lock()==EOK) {
        while(!data.b2_rdy) {
            printf("Consumer 2 going to sleep\n");
            pthread_sleepon_wait( &data.b2_rdy );
            printf("Consumer 2 - exiting sleep\n");
        } // end inner while
        printf("Consumer2 got: %c\n", data.byte2);
        pthread_sleepon_unlock();
    } // end if
}
```

```
void main( void ) {
    pthread_create(NULL, NULL, consumer1_ex, NULL);
    pthread_create(NULL, NULL, consumer2_ex, NULL);
    pthread_create(NULL, NULL, producer_ex, NULL);
    sleep(20); // let threads run
    printf("Main Terminated...\n");
}
```

Sleepon Locks – Example 2 RMIT Classification: Trusted used with Loops

Added while(1) loop. Please try the code by yourself:

```
#include <stdlib.h>
#include <pthread.h>
#include <errno.h>

struct resource {
    volatile char byte1, byte2;
    volatile char b1_rdy, b2_rdy; } data;

void *producer_ex(void *notused) {
    char buf[3]={};
    while(1) {
        printf("Type 2 chars (a-z):\n");
        strcpy( buf, "  "); // clear buffer
        gets(buf);           // read keyboard
        if( pthread_sleepon_lock() == EOK ) {
            data.byte1 = buf[0]; // key value
            data.byte2 = buf[1]; // modified
            data.b1_rdy = 1;     // make true
            data.b2_rdy = 1;     // make true
            // send signal to blocked threads
            pthread_sleepon_signal(&data.b1_rdy);
            pthread_sleepon_signal(&data.b2_rdy);
            //pthread_sleepon_broadcast(&data);
            pthread_sleepon_unlock();
        } // end if
    } // end while
}
```

```
void *consumer1_ex(void *notused) {
    while(1) {
        if(pthread_sleepon_lock()==EOK) {
            while(!data.b1_rdy) {
                printf("Consumer 1 going to sleep\n");
                pthread_sleepon_wait( &data.b1_rdy );
                printf("Consumer 1 - exiting sleep\n");
            } // end inner while
            printf("Consumer 1 got: %c\n", data.byte1);
            data.b1_rdy = 0; // make false
            pthread_sleepon_unlock();
        } // end if
        else printf("Sleeplock ERROR in C1\n");
    } // end outer while
}
```

Second Consumer Thread
NOT shown (copy this one
and replace 1 with 2)

```
void main( void )
{
    pthread_t th1, th2, th3;
    pthread_create(&th1, NULL, consumer1_ex, NULL);
    pthread_create(&th2, NULL, consumer2_ex, NULL);
    pthread_create(&th3, NULL, producer_ex, NULL);

    pthread_join (th1, NULL);
    pthread_join (th2, NULL);
    printf("Main Terminated...\n");
}
```

Conditional Variable (also known as a *Condvar*)

- The `pthread_cond_wait()` function blocks the calling thread on the condition variable *cond*, and unlocks the associated *mutex*. The calling thread must have locked *mutex* before waiting on the condition variable.

```
#include <pthread.h>

int pthread_cond_wait( pthread_cond_t* cond, pthread_mutex_t* mutex );
```

- *cond* is a pointer to the `pthread_cond_t` object that you want the thread to block on
- *mutex* is the *mutex* that you want to unlock
- If a deadline must be met, there is a timed wait condition version of the function:

```
int pthread_cond_timedwait( pthread_cond_t* cond, pthread_mutex_t* mutex,
                           const struct timespec* abstime );
```

- To specify the clock source for the condvar, call `pthread_condattr_setclock()`.
 - Can be `CLOCK_REALTIME` (the default) or `CLOCK_MONOTONIC`
- To unblock a thread that's waiting on a condition variable, use:

```
int pthread_cond_signal( pthread_cond_t* cond );
Or
int pthread_cond_broadcast( pthread_cond_t* cond );
```

- `signal()` function unblocks the highest priority thread that's waiting on the condition variable.
- `broadcast()` function unblocks all threads currently blocked on the condition variable *cond*. The threads are unblocked in priority order. If more than one thread at a particular priority is blocked, those threads are unblocked in FIFO order.

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_procs_condvar.html

Condvar Usage

- Generic usage form of a Condvar:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int Condition_variable = 1; //shared var (ctrl)
int data = 0;                //shared var (data)

void * thread_ex(void*) {
    // .. Other code

    pthread_mutex_lock(&mutex );
    while( Condition_variable == 1 ) {
        pthread_cond_wait(&cond, &mutex); // Block here and release mutex
    }
    // .. Do what ever work we need to do
    data = 123;

    Condition_variable = 1; // Set the condition variable to force the
    // above loop to execute. This is done so the other thread gets priority
    pthread_cond_signal(&cond); // Signal other thread check condition
    pthread_mutex_unlock(&mutex);

    // .. Other code
}
```

- Both the variable **cond** and associated **mutex** object need to be initialised before use

Condvar Example 1

Producer data then signals consumer.

```
void consume( void )
{
    while( 1 )
    {
        pthread_mutex_lock(&mutex );

        while(ProduceData == 1 ) {
            printf(">--> Waiting In Consumer\n");
            pthread_cond_wait(&cond, &mutex);
        }
        printf("Consumed %d\n", count);
        ProduceData=1; // Stop consuming data
                        // and signal consumer
        usleep(500); // wait 0.0005 seconds
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
        //sched_yield(); // used to force
                        // priority to
                        // producer thread
    }
}

void main( void )
{
    pthread_create(NULL, NULL, &produce, NULL);
    consume(); // producer must start first
}
```

```
#include <stdio.h>
#include <pthread.h>
```

```
pthread_mutex_t mutex =
    PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond =
    PTHREAD_COND_INITIALIZER;
```

```
int ProduceData = 1; //shared var (ctrl)
int count = 0;       //shared var (data)
```

```
void *produce( void * arg )
{
```

```
    while( 1 )
    {
```

```
        pthread_mutex_lock(&mutex);
```

```
        while( ProduceData == 0 ) {
            printf("<-Waiting In Producer\n");
            pthread_cond_wait(&cond, &mutex);
        }
```

```
        printf("Produced %d\n", ++count );
        ProduceData=0; // Stop producing data
                        //to allow consumer to current data
```

```
        pthread_cond_signal(&cond);
```

```
        pthread_mutex_unlock(&mutex);
```

```
        // sched_yield(); // Used to force
```

```
    }
    // priority to
    // consumer thread
}
```

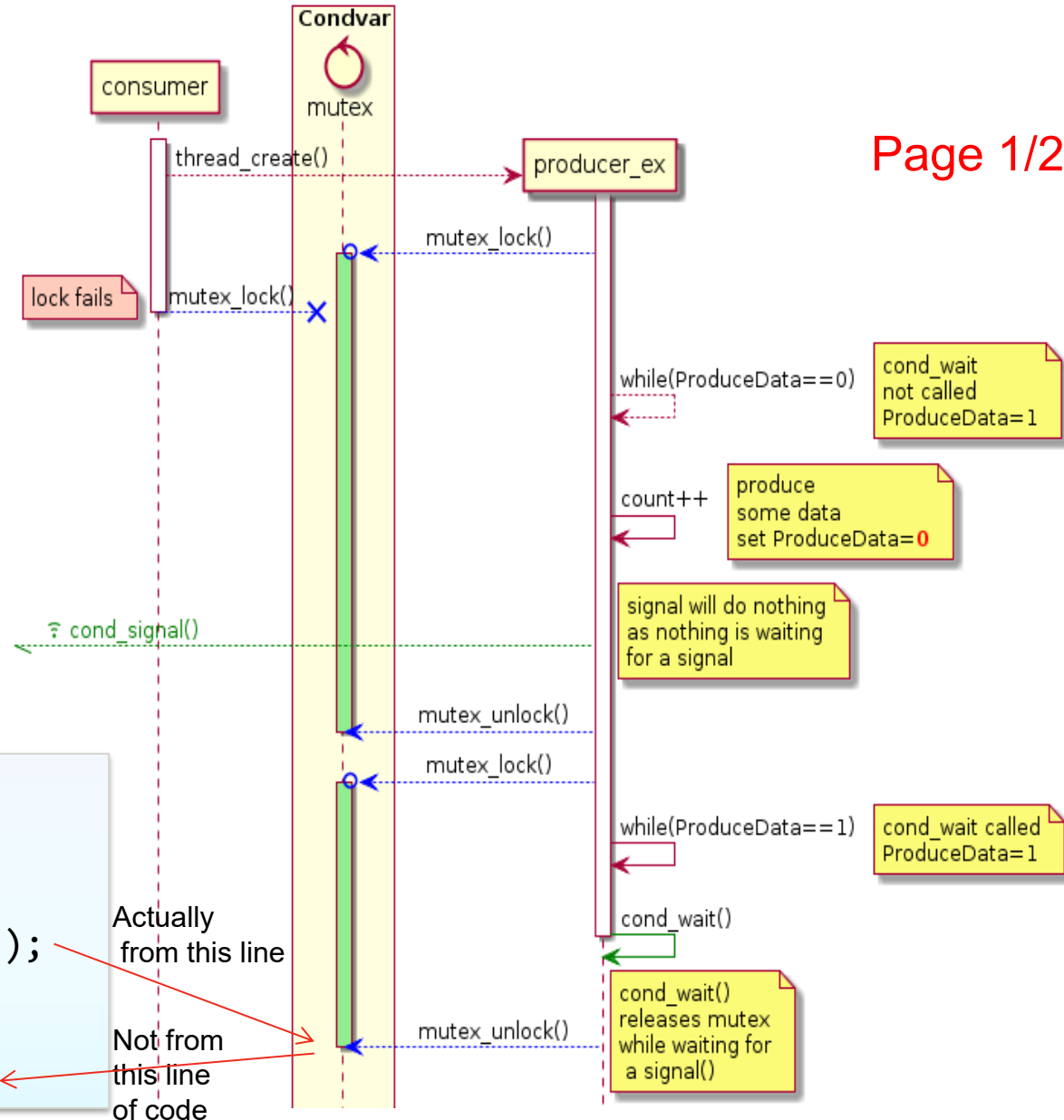
Condvar Example 1a (Sequence Diagram)

Producer produces data then signals consumer. The producer must stop producing data if the consumer does not use it.

The inner while loop control variable is used to control if `cond_wait()` function should be called. This will release the mutex until the other thread signals

The mutex is used to limit which thread is in the CS

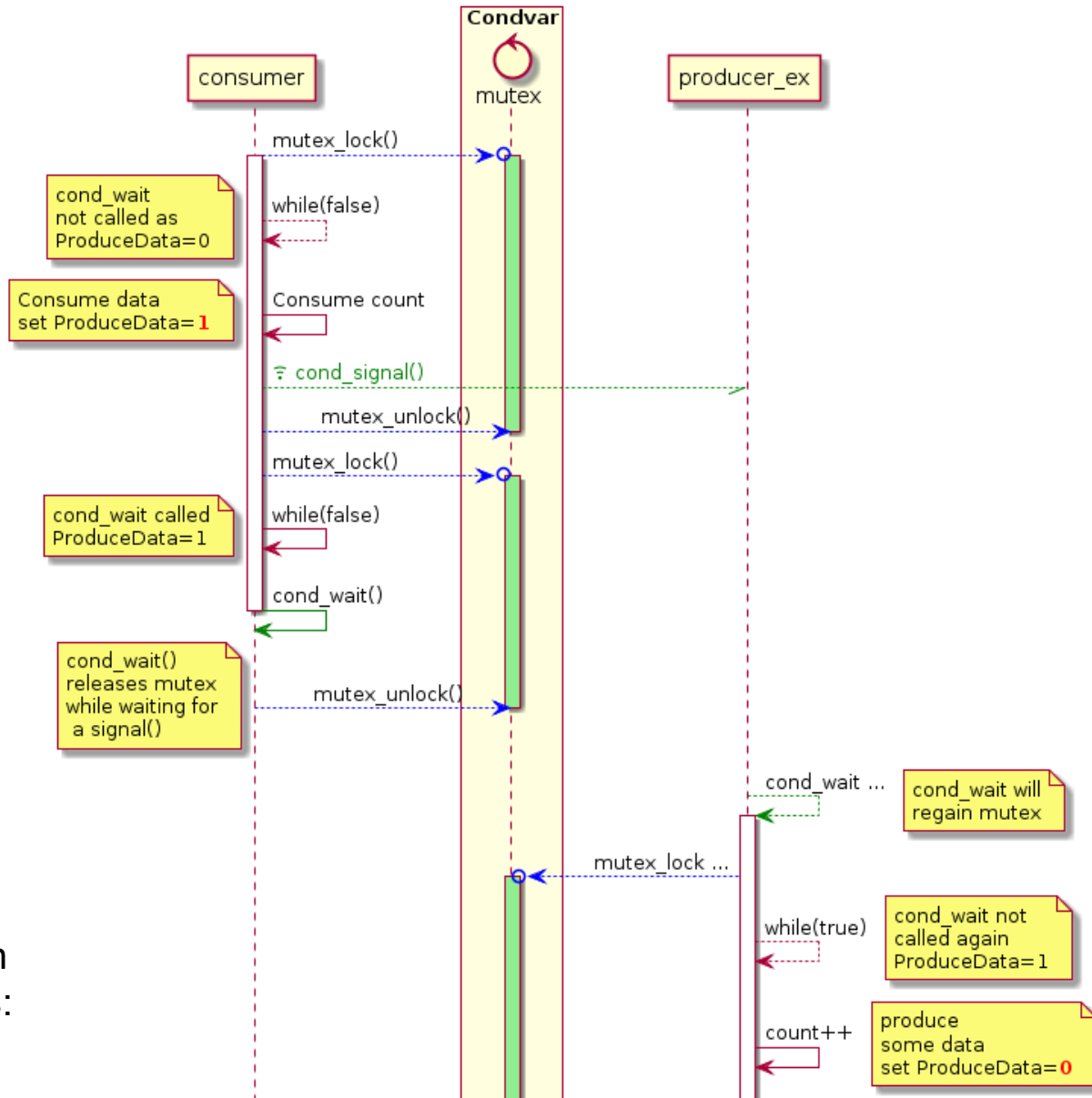
```
pthread_mutex_lock( &m );
// . . .
while(!arbitrary_condition) {
    pthread_cond_wait( &cv, &m );
}
// . . .
pthread_mutex_unlock( &m );
```



Page 1/2

Condvar Example 1a (Sequence Diagram cont.)

Page 2/2



See sample code with extra printf statements:

Condvar_printf.c

Condvar Example 1b

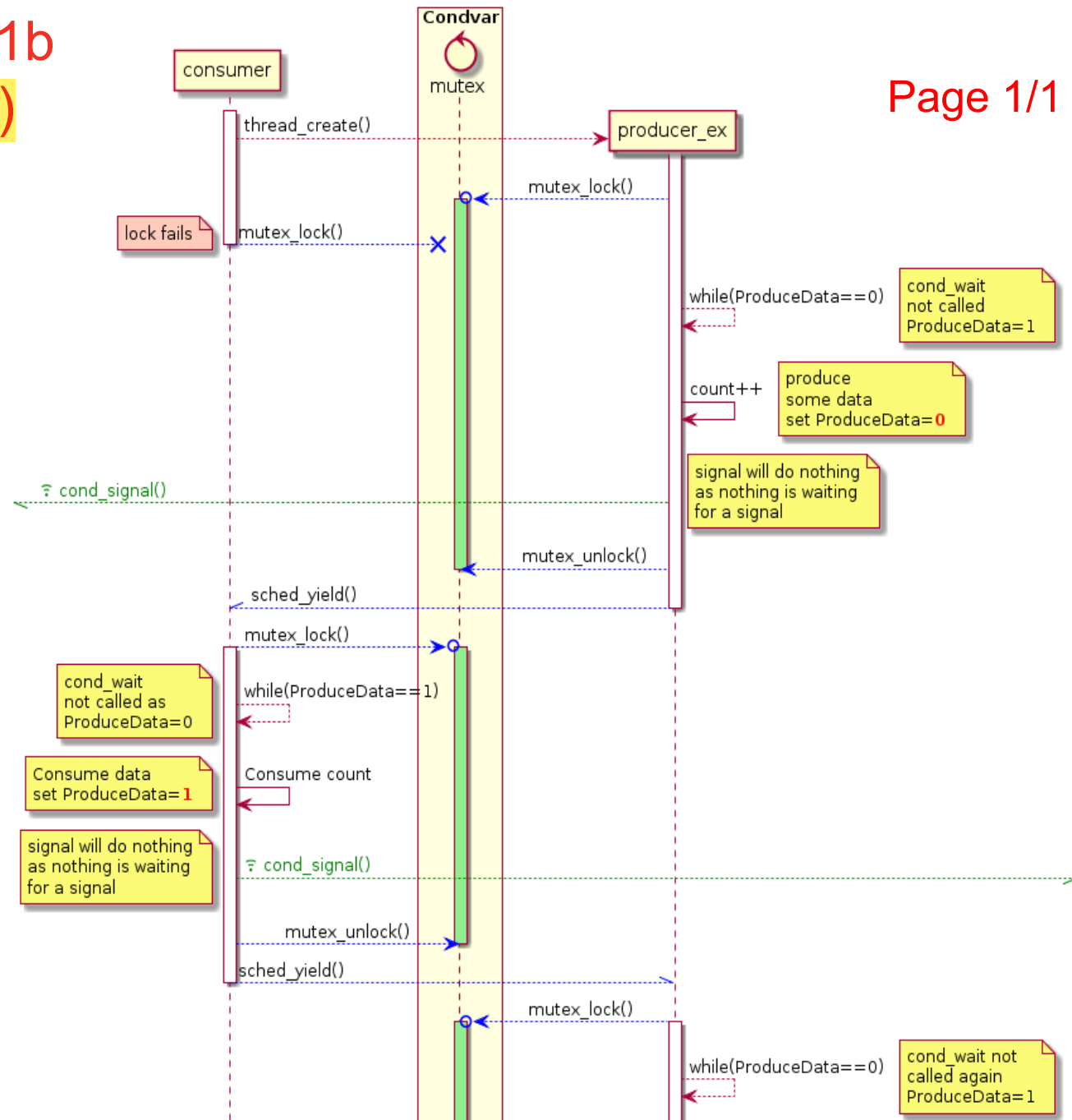
using sched_yield()

Page 1/1

Here the `sched_yield()` is used by both the consumer and producer to pass over priority to the other thread.

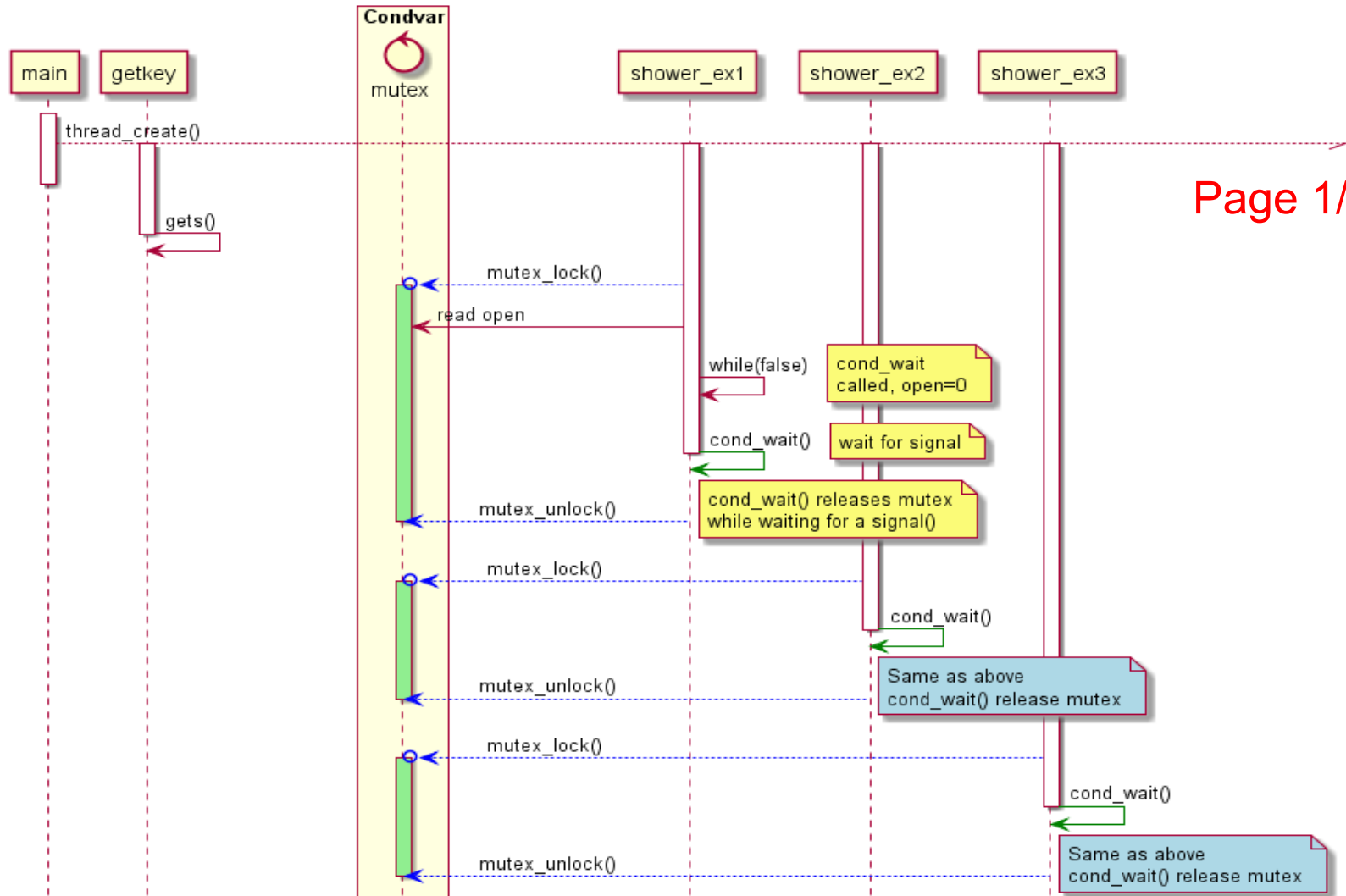
This eliminates the `cond_wait()` function being called as the while loop does not execute due to each thread changing the condition variable.

Producer thread must still obtain the mutex first so valid data can be created.



Condvar Example 2 – Single Bathroom, Multiple people

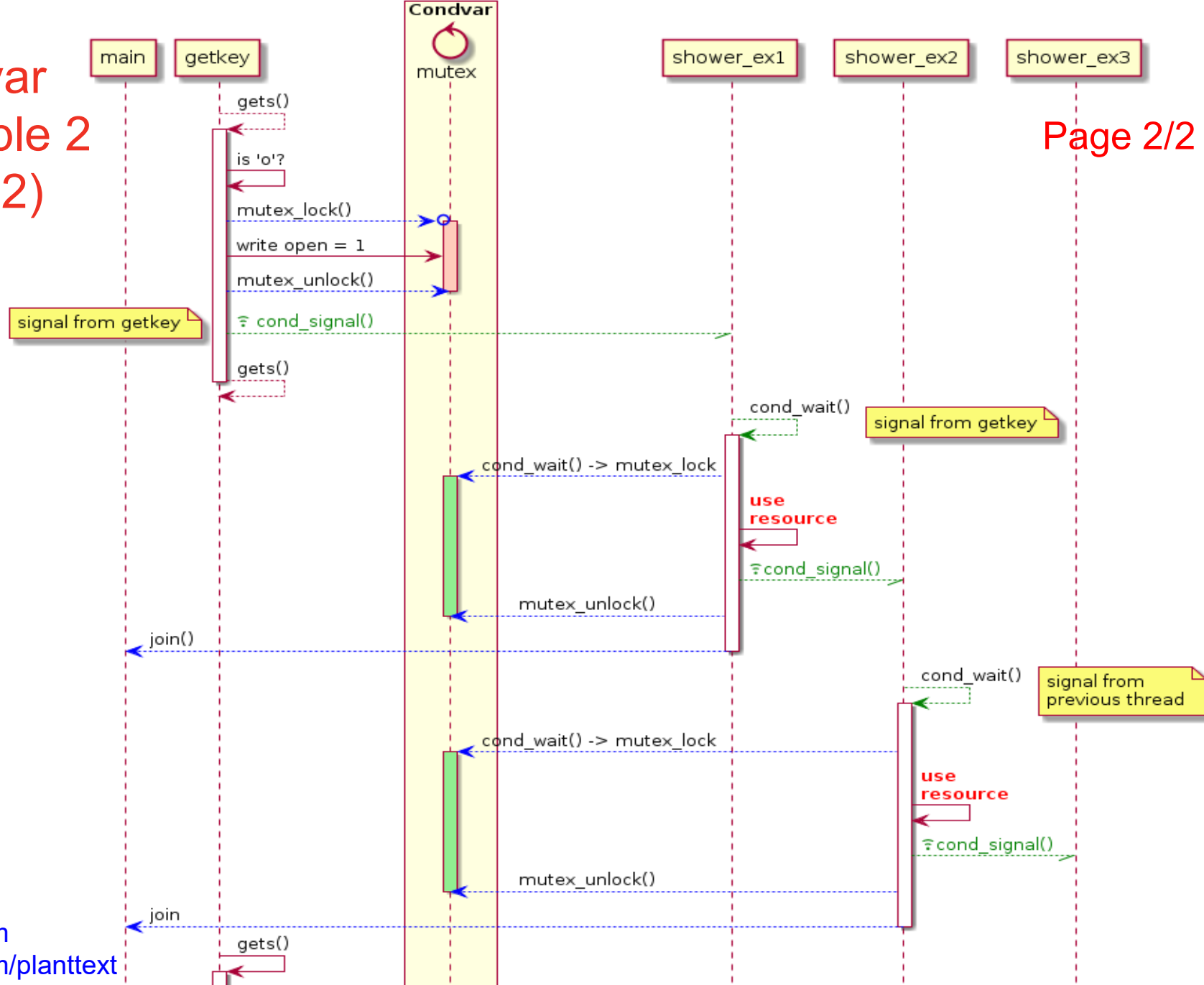
Multiple consumer threads (showers), need to use the bathroom. Only one person at a time can use the resource. However the condition is that it **must be open for use** **getkey** thread is waiting for the character 'o' before setting the condition to open.



Page 1/2

Condvar Example 2 (page 2)

Page 2/2



Condvar Example 2

- Program launches the keyboard reader thread and N number of shower threads.
- Only one shower resource can be used at a time
 - All other shower threads block until they have exclusive access to the critical section

```
void main( void )
{
    printf("Bathroom Example - Convar\n\n");
    printf(" Instructions: Press 'o' to open the bathroom\n");
    printf("          Press any other key to close it!\n");

    pthread_t th_Key;          // start key scanning thread
    pthread_create(&th_Key, NULL, getkey_ex, NULL);

    pthread_t *thread_ids;     // create variables for thread IDs
    thread_ids = malloc(sizeof (pthread_t) * N);

    int threads_launched = 0;
    while(threads_launched < N)
    {
        // create the shower threads (N threads)
        pthread_create(&thread_ids[threads_launched], NULL, shower_ex, NULL);
        threads_launched++;
    }

    int thread;                // synchronize to termination of all threads
    for(thread = 0; thread < threads_launched; thread++)
        pthread_join(thread_ids[thread], NULL);

    printf("Main Terminated...\n");
}
```

Condvar Example 2

- The calling thread is blocked until another thread performs a signal or broadcast on the condition variable.
 - Don't use a recursive mutex with condition variables for this reason

```
#include <stdlib.h>
#include <pthread.h>

#define N 5 // Number threads to create
int open = 0; // Showers closed
```

```
pthread_mutex_t someoneisbathing = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condvar = PTHREAD_COND_INITIALIZER;
```

```
void *shower_ex(void *notused)
{
```

```
    pthread_mutex_lock(&someoneisbathing);
```

```
    while(open==0) // test the condition and wait until it is true
```

```
        pthread_cond_wait(&condvar, &someoneisbathing); // Block on cond variable, and
                                                         // unlock the associated mutex
```

```
    // this is the critical section:
```

```
    printf("Thread %d is using shower\n", pthread_self());
```

```
    sleep(2);
```

```
    printf("Thread %d is leaving shower\n", pthread_self());
```

```
    pthread_cond_signal(&condvar); // Signal for next thread to run (unless broadcast)
```

```
    pthread_mutex_unlock(&someoneisbathing); // used in the getkey_ex thread
```

```
}
```

```
RMIT C void *getkey_ex(void *notused) {
    char key[10] = {};
    while(1) {
        gets(key);
        if(key[0]=='o') {printf("-> Shower Open!\n");
            open = 1; // set variable for the condvar
        }
        else { printf(" <-- Showers Closed!\n");
            open = 0; // close it
        }
        pthread_cond_signal(&condvar);
    }
    sched_yield();
}
```

main() on previous slide

Any question so far?

Semaphores

- Unlike **mutex** objects, **semaphores** can have a **count greater than 1**
 - A semaphore with a positive value will not block
 - A semaphore with a value of zero will block until another process executes a “**post**”
 - A semaphore cannot have a negative value
- Semaphores are “**async safe**” and can be **manipulated by signal handlers**
 - If the desired effect is to have a signal handler wake a thread, semaphores are a good choice, however there are some downsides as they are more resource intensive.
- Major benefit: Semaphores can also **work across nodes** (i.e. names semaphore)
 - **Unnamed semaphores** are implemented **as kernel primitives** (in QNX).
 - **Named semaphores** require that a separate named-semaphore manager is running.
 - Takes place on top of standard message-passing.
 - Is far more resource and time costly than an unnamed variant, which in turn is more costly than *Mutex* or *Condvar* equivalents.
- Semaphores don't affect a thread's priority
 - if you need priority inheritance, use a *mutex* or *Condvar*.
- Semaphores can be used as either a **binary** semaphore or a **counting** semaphore. You can set this behaviour at initialisation time



[3]

See: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_procs_semaphore.html

Unnamed Semaphores in QNX

- To initialise an unnamed semaphore, use:

```
#include <semaphore.h>

int sem_init( sem_t * sem, int pshared, unsigned value );
```

- **sem** - A pointer to the **sem_t** object for the semaphore to be initialized (32-bit aligned)
- **pshared** - NULL or nonzero if it will be used between processes via shared memory.
- **value** - The initial value of the semaphore. Zero = locked. A positive value (i.e., greater than zero) indicates an unlocked semaphore. must not exceed **SEM_VALUE_MAX**

- To wait on (decrement) a named or unnamed semaphore

```
int sem_wait( sem_t * sem );
```

- If the semaphore value is not greater than zero, then the calling process blocks until it can decrement the counter, or the call is interrupted by signal.

- To post (increment) a named or unnamed semaphore

```
int sem_post( sem_t * sem );
```

- One of the currently blocked thread waiting for the semaphore will be unblocked.
 - Thread to be unblocked is determined in accordance with the scheduling policies
 - The highest priority waiting thread is unblocked, and if there is more than one highest priority thread blocked waiting for the semaphore, then the highest priority thread that has been waiting the longest is unblocked.

- To destroy an unnamed semaphore once you finished using it:

```
int sem_destroy( sem_t * sem );
```


Named Semaphores in QNX

- To create or access a named semaphore

```
#include <semaphore.h>
#include <fcntl.h>

sem_t * sem_open( const char * sem_name, int oflags, mode_t mode, unsigned int value );
```

- **sem_name** - name of the semaphore that is to be created or accessed
 - **oflags** - Flag that affect how the a new semaphore will b created. This argument is a combination of: O_CREAT and/or O_EXCL
 - **mode** – Defines read, write, and execute mode (like files): S_IRWXG for group access; S_IRWXO for other's access; S_IRWXU for your own access.
 - **value** –T he initial value of the semaphore. Zero = locked. A positive value (i.e., greater than zero) indicates an unlocked semaphore. must not exceed **SEM_VALUE_MAX**
 - Once the semaphore descriptor has been returned by **sem_open()**, you can use with **sem_wait()**, **sem_trywait()**, and **sem_post()**.
- To try wait on a named or unnamed semaphore, but **NOT block**:

```
int sem_trywait( sem_t * sem );
```

- To destroy a named semaphore (call before **sem_close()**):

```
int sem_unlink( const char * sem_name );
```

- Any attempt to use **sem_open()** on an unlinked semaphore will create a *new* semaphore. Semaphores are persistent as long as the system remains up.
- To release any system resources associated with the named semaphore:

```
int sem_close( sem_t * sem );
```


Mutual Exclusion with (Unnamed) Binary Semaphore

- Example creates 5 threads that take it in turn to use the semaphore controlled CS.
 - Only one thread at a time can be in the critical section.
 - Semaphore only has a maximum value of 1.

```
void main( void )
{
    sem_init(&someoneisbathing, NULL, 1);

    // create variables for thread IDs
    pthread_t *thread_ids;
    thread_ids = malloc( sizeof(pthread_t) * N);

    int threads_launched = 0;
    while(threads_launched < N) { // create the shower threads (N threads)
        pthread_create(&thread_ids[threads_launched], NULL, shower_ex, NULL);
        threads_launched++;
    }

    int thread; // synchronize to termination of all threads
    for(thread = 0; thread < threads_launched; thread++)
        pthread_join(thread_ids[thread], NULL);

    printf("Main Terminated...\n");
}
```

RMIT Classification

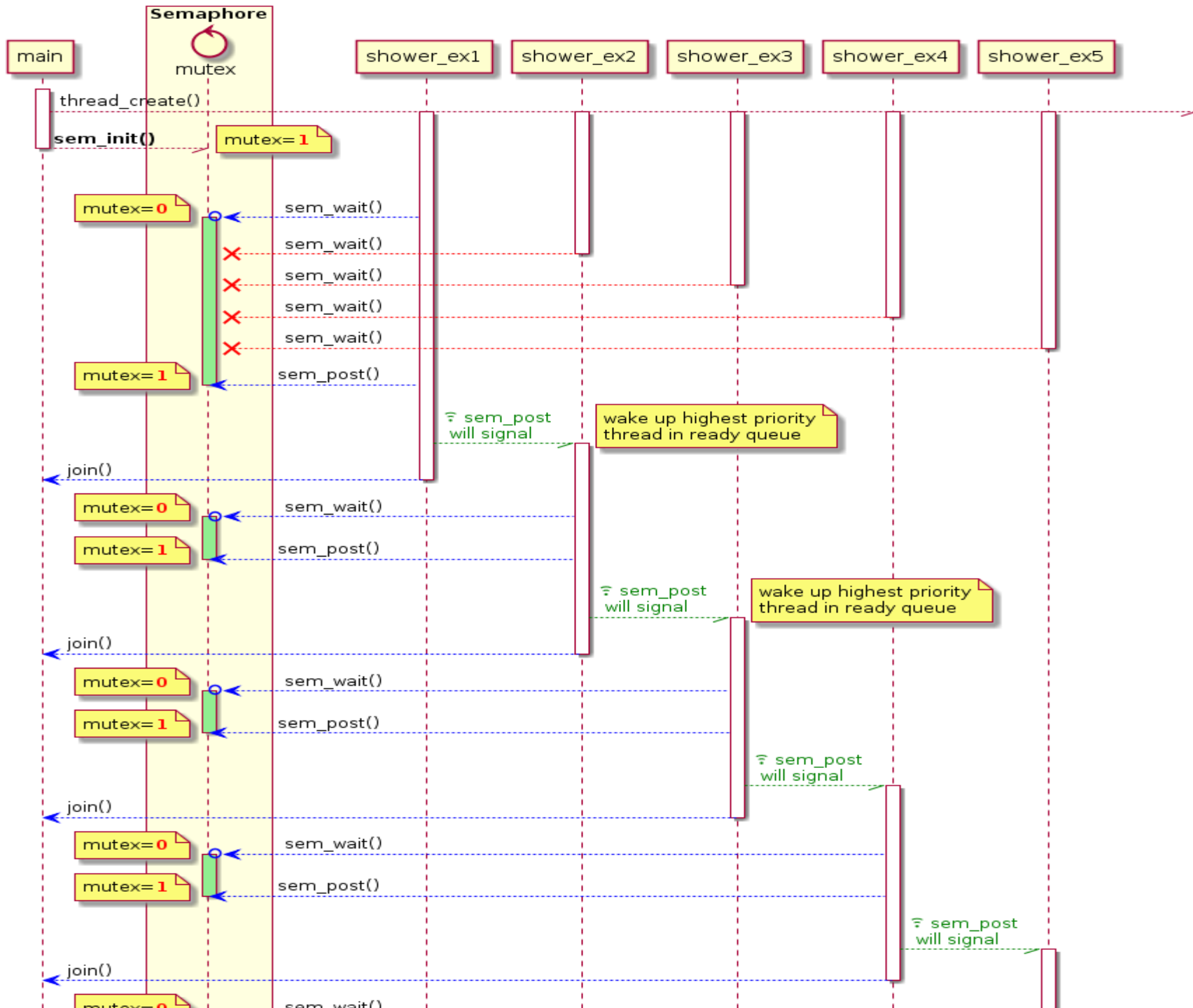
```
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5 // Number threads to create

sem_t someoneisbathing;

void *shower_ex(void *notused)
{
    int ID = pthread_self();
    sem_wait(&someoneisbathing);
    printf("Thread %d is using shower\n", ID);
    sleep(2);
    printf("Thread %d is leaving\n", ID);
    sem_post(&someoneisbathing);
}
```

RMIT Un



Limitations of Semaphores

- The definition of semaphores does not guarantee fairness
 - in particular:
 - If several processes simultaneously perform a down on a semaphore, any one of them could be given the semaphore, and the rest will be blocked.
 - Also if several processes are blocked waiting on a semaphore, any one of them could be woken-up by a up on that semaphore.
 - QNX wakes the highest priority waiting thread
 - If more than one thread is waiting and they all have the same priority, then thread that has been waiting the longest is unblocked.
- Using semaphores does not guarantee deadlock and livelock will be avoided
 - If used correctly, deadlock and livelock should not occur
 - For example for every `sem_wait()`, a `sem_post()` should be called.
- In general, mutexes are much faster than semaphores. A semaphore always require a kernel entry.
- Semaphores don't affect a thread's effective priority
 - If you need priority inheritance, use a mutex or condvar (or message passing).
 - If you need priority inheritance across nodes then the only option is to use native QNX Message passing as a synchronization method.

Mutual Exclusion with (Un-named) Counting Semaphore

- Example creates 30 threads.
 - Only 5 thread at a time can be in the critical section.
 - Semaphore only has a maximum value of 5.

```
void main( void )
{
    sem_init(&peoplebathing, NULL, spots);
    // create variables for thread IDs
    pthread_t *thread_ids;
    thread_ids = malloc( sizeof(pthread_t) * N);
    int threads_launched = 0;
    while(threads_launched < N) { // create the shower threads (N threads)
        pthread_create(&thread_ids[threads_launched], NULL, shower_ex, NULL);
        threads_launched++;
    }

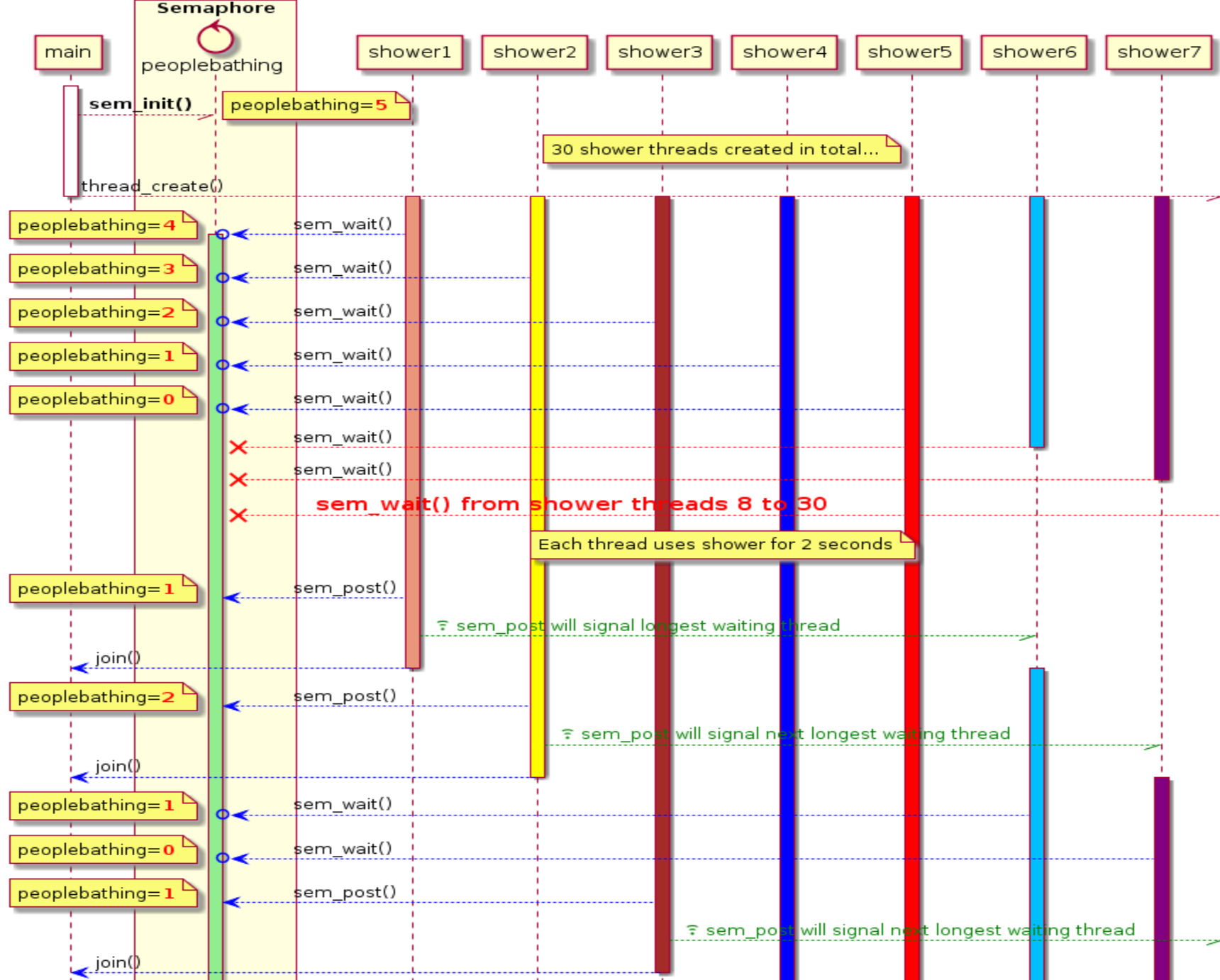
    int thread; // synchronize to termination of all threads
    for(thread = 0; thread < threads_launched; thread++)
        pthread_join(thread_ids[thread], NULL);
    printf("Main Terminated...\n");
}
```

```
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define N 30 // Number threads to create
#define spots 5 // Number of showers spots

sem_t peoplebathing;

void *shower_ex(void *notused) {
    int ID = pthread_self();
    int count = 0;
    sem_wait(&peoplebathing);
    sem_getvalue( &peoplebathing, &count);
    printf("---> Thread %d is using shower, "
           "spots full=%d\n", ID, spots-count);
    sleep(2); // spend time in the shower
    sem_post(&peoplebathing);
    sem_getvalue( &peoplebathing, &count);
    printf("<-Thread %d finished showering, "
           "spots full=%d\n", ID, spots-count);
}
```

Mutual Exclusion with Counting Semaphore



Block/Wakeup Process Synchronization using Semaphores

- A semaphore can be used to block a process until a condition becomes true
 - Similar to the functionality of a Sleepon Lock or Condvar.
 - However if it is a named semaphore, this can be used to synchronize processes that are operating on 2 or more nodes

```
sem_t EventOfInterest = 0;
    // use sem_init()

void * ProcessOne(void *arg)
{
    //preliminarystuffone
    sem_wait(&mutex);
    //otherstuffone
}
```

```
void * ProcessTwo(void *arg)
{
    //preliminarystufftwo
    sem_post(&mutex);
    //otherstufftwo
}
```

- Here **ProcessOne** can only do **otherstuffone** after **ProcessTwo** has completed **preliminarystufftwo**
 - Far more costly than a Sleepon Lock (computation wise) however has greater flexibility
 - Similar functionality can also be accomplished with native QNX Message passing

Readers and Writers using Semaphores

- Semaphore can also be used to provide reader and writer locks
 - Multiple readers and writers by using counting semaphores
 - 1 semaphore for controlling readers
 - 1 semaphore for controlling the writing processes

Structure of a writer process

```
//global variables
sem_t mutex = 1;
sem_t wrt = 1;
int readcount = 0;
void * Writer(void *arg)
{
    //preliminarystuff
    sem_wait(&wrt);
    // write code goes here
    sem_post(&wrt);
    //otherstuff
}
```

Structure of a reader process

```
void * Reader(void *arg)
{
    //preliminarystuff
    sem_wait(&mutex);
    readcount++;
    if(readcount == 1)
        sem_wait(&wrt); //Block wrt
    sem_post(&mutex); //Allow other
                        //readers in

    // read code goes here

    sem_wait(&mutex);
    readcount--;
    if(readcount == 0)
        sem_post(&wrt);
    sem_post(&mutex);
    //otherstufftwo
}
```

- Structure allows multiple active readers (of some shared data) but only one active writer and no active readers when a writer is active.

Bounded Buffer with Semaphores

- Instead of using a counter with a **mutex** and **condvar** (like in lab 3), **two semaphore objects** can be used to maintain the state of the buffer and a 3rd can be used to provide exclusive access to the critical section in each thread (i.e. **producer** and **consumer** threads).

```
#define N 5           //buffer size

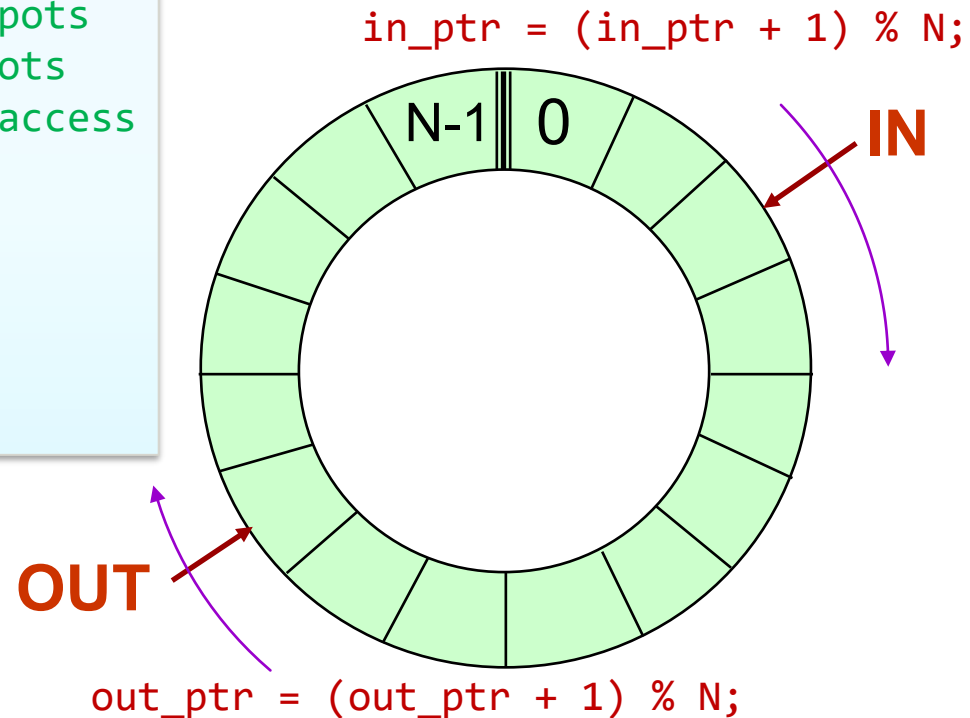
unsigned int buffer[N] = {};

sem_t empty = N; // number of empty spots
sem_t full = 0; // number of full spots
sem_t mutex = 1; // critical section access

int in_ptr = 0; // buffer in
int out_ptr = 0; // buffer out

int count = 0; // number of data
               // bytes in buffer
```

- The **mutex** is a binary semaphore
- full** and **empty** are counting semaphores (value can be > 1)



Bounded Buffer with Semaphores

- The base code for the producer and consumer threads would look like:

```
void * producer( void *arg) {
    int item =0;
    int in_ptr=0;
    while(1) {
        produce_item(&item);
         sem_wait(&empty);
         sem_wait(&mutex);
        buffer[in_ptr] = item;
        in_ptr = (in_ptr + a) % N;
        sem_post(&mutex);
        sem_post(&full);
    }
}
```

```
void * consumer( void *arg) {
    int item =0;
    int out_ptr=0;
    while(1) {
        sem_wait(&full);
        sem_wait(&mutex);
        item = buffer[out_ptr];
        out_ptr = (out_ptr + a) % N;
        sem_post(&mutex);
        sem_post(&empty);
        consume_item(item);
    }
}
```

- Does it make sense that the **mutex object** is created as a semaphore?
- Is the order in which the semaphore **wait** and **post** functions are called important?
 - What would happen if the **&empty** and **&mutex** lines were reversed in the producer?
 - Yes: **Deadlock will occur** if the producer does the **wait(&mutex)** when the mutex value is 1 (i.e. the consumer is not in the critical section) and when the buffer is full.

Bounded Buffer with Semaphores / Mutex

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5          // buffer size

int in_ptr =0;      // pointer to buffer in
int out_ptr =0;     // pointer to buffer out
int count=0;        // number of bytes in buffer
int i=0;            // index to get data

sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

unsigned int buffer[N] = {}; // actual buffer
unsigned int data[] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20}; // data source

void main( void )
{
    printf("Circular buffer Example\n");
    sem_init(&empty,NULL,N); // init sem buff size
    sem_init(&full,NULL,0);  // init sem to 0
    pthread_t  th1, th2;
    int t1 = 1000, t2 = 2000;
    pthread_create(&th1, NULL, producer_ex, &t1);
    pthread_create(&th2, NULL, consumer_ex, &t2);
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    printf("## main terminating...\n");
}

```

Multi thread ring buffer example:

- Uses mutex and counting Semaphores
- Buffer size of 5
- Data size of 20
- Speed of producer thread: 1msec
- Speed of consumer thread: 2msec

Bounded Buffer with Semaphores / Mutex

- Multi thread ring buffer example
 - Synchronisation provided by a **mutex**
 - Buffer availability provided by **semaphore**
 - Both are essential to the safe operation of the ring buffer

```
void *producer_ex(void *time) {
    int i = 0, tmp = 0;
    int *t = (int*)time;
    while(1)
    {
        if(i>19) break; // no more data to send
        tmp = data[i++]; // generate some data
        ➡ sem_wait(&empty); //if(count==N)
        ➡ pthread_mutex_lock(&mutex);

        buffer[in_ptr] = tmp; //wrt data buffer
        in_ptr = (in_ptr + 1) % N;
        count++;
        printf("-->producer inserted: %i", tmp);
        printf(" count=%i\n", count);
        ➡ pthread_mutex_unlock(&mutex);
        ➡ sem_post(&full);

        usleep(*t);
    }
    printf("## produce terminating...\n");
}
```

```
void *consumer_ex(void *time) {
    int i = 0, tmp = 0;
    int *t = (int*)time;
    while(1)
    {
        if(i>19) break; // no more data to read
        i++;

        ➡ sem_wait(&full);
        ➡ pthread_mutex_lock(&mutex);
        tmp = buffer[out_ptr]; //read data buffer
        out_ptr = (out_ptr + 1) % N;
        count--;
        printf("<--consumer removed: %i", tmp);
        printf(" count=%i\n", count);
        ➡ pthread_mutex_unlock(&mutex);
        ➡ sem_post(&empty);

        usleep(*t);
    }
    printf("## consumer terminating...\n");
}
```

What is the difference between this code and your code in lab 3?

Message Passing used for Synchronisation

- Send/Receive/Reply message-passing IPC services implement an implicit synchronization by their blocking nature.
- More advanced implementations can use the content of the message to control access by maintaining a linked list of waiting processes, and a **counter** for each required semaphore

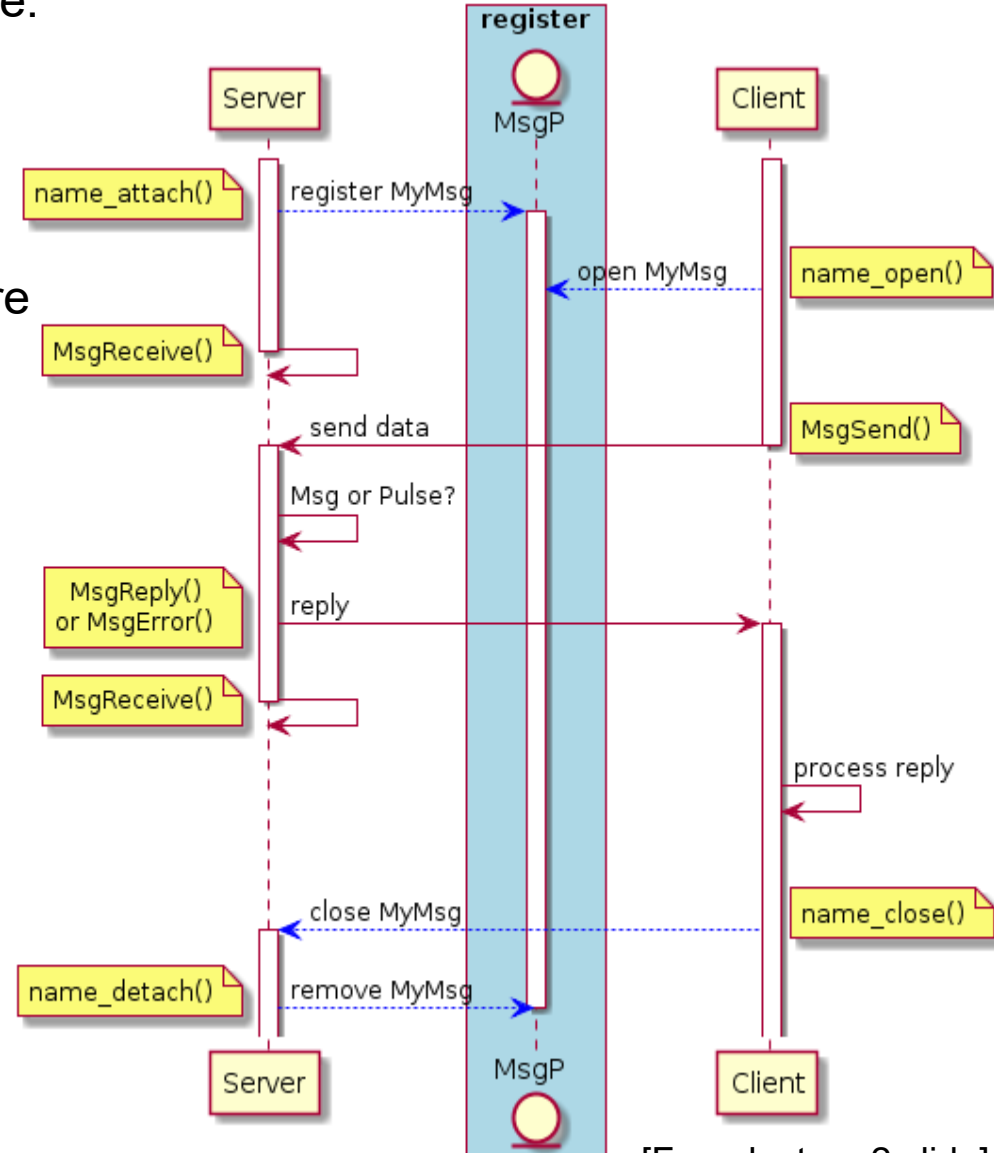
– `up( &sem )`

```
// msg = up, sem  
MsgSend(&synchproc, &msg);  
MsgReceive(&synchproc);
```

– `down( &sem )`

```
// msg = down, sem  
MsgSend(&synchproc, &msg);  
MsgReceive(&synchproc);
```

- Synchproc does not return a message to calling process on a down if count = 0 (hence blocks process) else returns an empty (null) message.
- On an up it returns an empty (null) message to a waiting process (if any)



[From lecture 2 slide]

Message Passing used for Synchronisation

- Pseudo code for both a producer (client) thread and a consumer (server) thread which use message passing to provide synchronisation between each other.

```
// server code

void * consumer( void *arg ) {

    name_attach() // create connection
                  // point (could be
                  // networked)

    MsgReceive() // block until msg recv

    // determine msg type (pulse, data)

    // if data msg
    MsgReply()
    // else - must be a pulse
    ConnectDetach() // disconnect
}
```

MsgPass_Semaphore.c

```
// client code

void * producer( void *arg ) {

    name_open() // try to connect to consumer

    loop for a number of times {
        // generate some data

        // send the data and block
        // until a reply is received
        MsgSend(data, replybuffer)

        // process the reply msg

        // loop
    }

    name_close() // send disconnect request
}
```

- The producer and consumer could be running on different nodes as Message passing works over networks with priority inheritance.

Synchronization Trade-offs

Ranking from worst to best choices in regards to computation efficiency:

- **Message Passing**
 - If you need everything a named Semaphore can provide, plus priority inheritance & more
- **Named Semaphore**
 - The extra messaging required to provide the named feature imposes extra overhead
 - This involves **procnto** and **qnet** process for network and pathname style location
 - A major advantage is that they can be easily used between processes.
- **Unnamed Semaphore**
 - The **sem_wait/sem_post** operations are managed by the kernel calls
 - This has a smaller overhead than message passing and server operations required for a named semaphore, but is still a kernel call for each semaphore call.
- **Condvar**
 - A **condvar** can be used as a more efficient counting semaphore as it relies on a **mutex** base. Can provide greater flexibility and potentially higher throughput, though requires more complex coding solution than a semaphore object.
- **Mutex**
 - A **mutex** does not provide the same semantics as a counting semaphore
 - Far better performance to a binary semaphore
 - **Mutexes** are optimized to use the processor's atomic operations, rather than kernel calls. Kernel is only required to intervene if the mutex is contested
- Overall, for best performance, the execution time of the **critical section should be small and of bounded duration – use mutex's and condvar's if you can**

Periodic Timers

Periodic Timers

- QNX implements POSIX timers. Timers can be periodic or single-shot.
 - QNX POSIX timers can send a standard UNIX style signal when they expire, or they can create a thread.
 - The kind of timer created is a combination of arguments used when invoking the `timer_settime()` function.
- The process of setting up a timer involves using communications channels to receive messages/pulses from the timer.
 - This removes the need for using `sleep()` type functions within a thread, rather your thread will block until it receives a message from the timer.

- To create a timer, use:

```
#include <signal.h>
#include <time.h>

int timer_create( clockid_t clock_id, struct sigevent * evp, timer_t * timerid );
```

- `clock_id` – is the clock source to be used. Can be one of these:
 - `CLOCK_REALTIME` The standard POSIX-defined clock.
 - `CLOCK_MONOTONIC` The clock that always increases at a constant rate and can't be adjusted.
 - `CLOCK_SOFTTIME` A clock that is only active when the processor *isn't* in a power-saving mode.
- `evp` – NULL, or a pointer to a `sigevent` structure containing the event that you want to deliver when the timer fires.
- `timerid` – A pointer to a `timer_t` object where the function stores the ID of the timer.

Periodic Timers

- When using the QNX timers you will need to define two main structures
 - `sigevent` *Structure that describes an event* - passed to `timer_create()`
 - For example to set up an event based on signal pulse set the attribute `sigeventName.sigev_notify=SIGEV_PULSE;`
 - Or you can use one of the MACRO functions:


```
SIGEV_SIGNAL_INIT(eventp, signo);
SIGEV_SIGNAL_CODE_INIT(eventp, signo, value, code);
SIGEV_SIGNAL_THREAD_INIT(eventp, signo, value, code);
SIGEV_PULSE_INIT(eventp, coid, priority, code, value);
SIGEV_UNBLOCK_INIT(eventp);
SIGEV_INTR_INIT(eventp);
SIGEV_THREAD_INIT(eventp, func, val, attributes);
```
 - `itimerspec` *contains two timespec structures* – passed to `timer_settime()`
 - The first `timespec` structure contains the time left before the timer expires.


```
itimerspecName.it_value.tv_sec    // The number of seconds
itimerspecName.it_value.tv_nsec  // The number of nanoseconds
```
 - The second `timespec` structure contains the timer's reload value


```
itimerspecName.it_interval.tv_sec
itimerspecName.it_interval.tv_nsec
```
- Calling the `timer_settime()` function effectively starts the timer:

```
int timer_settime(timer_t timerid, int flags, const struct itimerspec * value,
                  struct itimerspec * ovalue );
```


Periodic Timers

- Psudeo code for creating a periodic timer that starts a thread when timer expires:

```
void function( void *arg ) { // could be main()

    ChannelCreate(); // Create a communications channel (or named connection)

    ConnectAttach(); // Create a connection back to ourselves for
                    // the timer to send the pulse on

    // setup event struct (type of event/signal)
    event.sigev_notify = SIGEV_PULSE or SIGEV_SIGNAL, SIGEV_THREAD, etc..
    // (optional) use sigevent MACRO to fill in struct if necessary. E.g:
    SIGEV_THREAD_INIT(&eventp, &thread_to_start, &val_to_pas, &thread_attributes);

    timer_create() // Create the timer, binding it to the event (ie the connection)

    // set up the timer struct
    // initial timer values
    itime.it_value.tv_sec = 1; // 1 second
    itime.it_value.tv_nsec = 500000000; // 500 million nsecs = .5 secs
    // periodic timer values
    itime.it_interval.tv_sec = 1; // 1 second
    itime.it_interval.tv_nsec = 0; // 0 seconds

    timer_settime(); // and start the timer!
}
```

Creating Threads with a Periodic Timer

```

void main( void ) {
    struct sigevent    event;
    struct itimerspec  itime;
    timer_t            timer_id;
    int chid=0, data = 5;

    // Create a communications channel
    chid = ChannelCreate(0);
    // create a connection back to ourselves
    event.sigev_coid = ConnectAttach(ND_LOCAL_NODE, 0,
                                     chid, _NTO_SIDE_CHANNEL, 0);

    if(event.sigev_coid != -1) {
        event.sigev_notify = SIGEV_THREAD; // Make the event create a new thread (POSIX)
        // use the macro function to fill in event struct
        SIGEV_THREAD_INIT(&event, &thread_ex, &data, NULL); // thread details(name, val, att)

        // create the timer, binding it to the event
        if( timer_create(CLOCK_REALTIME, &event, &timer_id) == -1) {
            printf("Error: Couldn't create a timer\n");
            return; }

        // setup the timer (1.5s initial delay value, 1.5s reload interval)
        itime.it_value.tv_sec = 1;           // 1 second
        itime.it_value.tv_nsec = 500000000; // 500 million nsecs = 0.5 secs
        itime.it_interval.tv_sec = 1;        // 1 second
        itime.it_interval.tv_nsec = 100000000; // 100 million nsecs = 0.1 secs

        timer_settime(timer_id, 0, &itime, NULL); // Start the timer!
        sleep(10); // let the time run for a few seconds
    }
    else printf("Error: Couldn't ConnectAttach to self!\n");

    printf("Main Terminated...\n\n");
}

```

```

#include <stdio.h>
#include <time.h>
#include <sys/netmgr.h>
#include <sys/neutrino.h>

int val = 0; // global variable

void *thread_ex(void *data) {
    int d = *((int*)data); // cast passed data
    printf("thread %d with data=%d started. ",
           pthread_self(), d);

    printf(" val=%d\n", val++); // increment val
    sleep(1);
}

```

You'll want to be particularly careful with this one, because if you specify too short an interval, you'll be flooded with new threads! This could eat up all your CPU and memory resources!

References

Images and content used in this presentation can be found:

- [1] QNX Documentation - [Get_Programming_with_the_QNX_Neutrino_RTOS.pdf](#)
and QNX Documentation - [QNX_Neutrino_RTOS_System_Architecture.pdf](#)
- [2] <http://15418.courses.cs.cmu.edu/spring2013/article/27>
- [3] <http://www.lovesailing.net/sailing-tests/sailing-test/take-test/semaphore-flags/29/>

Sequence Diagrams created using plantUML http://plantuml.com/PlantUML_Language_Reference_Guide.pdf

On the web based editor made available at: <http://www.planttext.com/>

Timer/clock: http://www.qnx.net/developers/docs/6.4.0/neutrino/getting_started/s1_timer.html

Note: Information on the QNX functions can be found by typing name in to the search box in the following link:

http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topics/s1_procs_More_synchronization.html

<http://www.qnx.com/developers/docs/7.0.0/>

BlackBerry's software used in more than 60 million cars is a target for the CIA to hack: WikiLeaks



- WikiLeaks release documents indicating that the CIA have identified QNX as one of several “potential mission areas” for the organization’s Embedded Devices Branch, similar to the CIA hacks targeted iPhones, Google’s Android system and Samsung smart TVs.
- QNX auto software is used in 60 million vehicles around the world.
 - It recently announced a \$100-million investment into an autonomous driving and the fact that QNX has very long history in the secure automotive systems.
- BlackBerry were keen to point out that QNX wasn’t involved in the recent Jeep Hacking, which showed that control could be taken away from the driver while on the highway.
 - That hack led to Fiat and Chrysler recalling 1.4 million cars and trucks – the first auto recall prompted by hacking fears.

[Source: <http://www.financialpost.com/m/wp/fp-tech-desk/blog.html?b=business.financialpost.com/fp-tech-desk/blackberry-ltds-software-used-in-more-than-60-million-cars-is-a-target-for-the-cia-to-hack-wikileaks>]

Acknowledgment

I would like to Thank and Acknowledge the support of: Dr. *Samuel Ippolito*, the Course Coordinator of corresponding “EEET2166 /1262 Real-time Systems Engineering & Design” course from RMIT Melbourne.

Semaphores

- Classic definition:

- $P(s)$ or $down(s)$ or $wait(s)$ = $sem_wait()$ in POSIX & QNX

Pseudo code representations:

```
while(s <= 0) //cant be negative
    s = s - 1
```

```
if(s > 0)
    s = s - 1;
else
    block the process that called down (s);
```

- $V(s)$ or $up(s)$ or $signal(s)$ = $sem_post()$ in POSIX & QNX

```
s = s + 1
```

```
if(any processes are blocked waiting on s)
    wakeup one of them;
else
    s = s + 1;
```

- $down(s)$ and $up(s)$ are mutually exclusive

- but not necessarily $down(s1)$ and $down(s2)$

- Semaphore operations are executed atomically

- Atomic $\equiv$ indivisible $\equiv$ uninterruptable

- By this definition a semaphore value is zero if and only if exactly one process is in its critical section.

$down$ and up are used in “Modern Operating Systems” by Tanenbaum

Mutual Exclusion with Semaphores

- Generic operation for mutual exclusion operations:

```
typedef int semaphore; // semaphore is a
                        // special kind of int

semaphore mutex = 1;

void processone()
{
    while(1)
    {
        //preliminarystuffone;
        down(&mutex);    // like mutex_lock()
        criticalsectionone();
        up(&mutex);      // like mutex_unlock()
        //otherstuffone;
    }
}
```

```
void processtwo()
{
    while(1)
    {
        //preliminarystufftwo
        down(&mutex);
        criticalsectiontwo();
        up(&mutex);
        //otherstufftwo
    }
}
```

- To use a semaphore in place of a mutex, you must always **initialised it to 1 first**
 - For mutual exclusion the semaphore should only ever be of value 1 or 0
 - A binary semaphore should only ever have values 1 or 0
- Semaphores can be used to enforce mutual exclusion for any number of processes, not just two processes.